

REPORTE DE PRÁCTICA: EVALUACIÓN DE EXPRESIONES, INCREMENTOS Y PRECEDENCIA DE OPERADORES

Asignatura: Programación Estructurada
Estudiante: Aldo Ezequiel Rodriguez Godoy
Código: 224097617
Sección: D04
Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha: 6 de febrero del 2026

INTRODUCCIÓN

En la arquitectura del lenguaje C, la correcta construcción y evaluación de expresiones aritméticas constituye un pilar fundamental para el desarrollo de algoritmos predecibles y libres de errores lógicos. A diferencia de las matemáticas abstractas, la programación estructurada procesa las ecuaciones de manera secuencial y bajo reglas estrictas definidas por el estándar del lenguaje. Dos de los conceptos más determinantes y propensos a interpretaciones erróneas en este ámbito son los operadores de incremento y decremento unarios, así como la jerarquía o precedencia de operadores aritméticos.

Los operadores unarios de incremento (`++`) y decremento (`--`) permiten modificar el valor de una variable en una unidad. Sin embargo, su comportamiento semántico varía drásticamente dependiendo de su posición relativa respecto al identificador de la variable. Cuando se emplean como prefijos (ej. `--b`), el compilador ejecuta la alteración aritmética de manera inmediata en la memoria RAM y posteriormente utiliza el valor resultante en la expresión en curso. Por el contrario, cuando actúan como sufijos o post-fijos (ej. `a++`), el sistema evalúa la expresión utilizando el valor actual que posee la variable en ese instante y, únicamente tras haber resuelto dicha línea o sentencia de control, aplica el incremento físico en la memoria.

Adicionalmente, la evaluación de expresiones compuestas está regida por la precedencia y asociatividad de operadores. El lenguaje asigna prioridades numéricas a cada símbolo; por ejemplo, las operaciones multiplicativas (`*`, `/`, `%`) poseen un nivel de precedencia superior a las operaciones aditivas (`+`, `-`). Para alterar este orden natural de ejecución y forzar un cálculo específico, se recurre al uso de paréntesis asociativos `()`, los cuales ostentan la prioridad más alta dentro de la gramática de C. Comprender estos mecanismos a nivel de bits y registros de memoria es vital para evitar efectos secundarios imprevistos en el flujo de control del software.

OBJETIVO

Objetivo General

Diseñar, codificar e interpretar un programa de diagnóstico en lenguaje C para evaluar de forma experimental los mecanismos internos del compilador respecto a la precedencia de operadores unarios, binarios y la alteración de flujo mediante paréntesis asociativos.

Objetivos Particulares

- Demostrar de forma práctica la diferencia semántica y el orden de ejecución entre operadores de post-incremento (`a++`) y pre-decremento (`--b`) dentro de una misma sentencia aritmética.
- Validar el impacto de la jerarquía estándar de operadores mediante la comparación de una expresión aritmética mixta lineal frente a una expresión modificada jerárquicamente por signos de agrupación.
- Monitorear los cambios de estado de las variables en la memoria RAM imprimiendo los resultados formateados en la consola del sistema para comprobar las hipótesis matemáticas planteadas.

DESARROLLO

El desarrollo de la práctica se enfocó en la implementación de un algoritmo conciso pero de alta densidad conceptual dentro del archivo fuente principal `main.c`. El procedimiento se dividió en dos fases de experimentación controlada:

1. Experimentación con Operadores Unarios Combinados

En el primer bloque de código, declaré e inicialicé dos variables enteras primitivas: $a = 3$ y $b = 4$. Posteriormente, se introdujo una variable c asignándole el resultado de la expresión compuesta: $c = a++ + --b$;

Durante la ejecución, el compilador procesó la línea aplicando de manera rigurosa las reglas gramaticales de C:

- Al detectar el post-incremento `a++`, el sistema extrajo el valor actual de la variable (3) para ser usado de manera temporal en la operación de suma.
- Al detectar el pre-decremento `--b`, el compilador detuvo momentáneamente la evaluación de la suma, redujo el valor de `b` de 4 a 3 directamente en su celda de memoria, y tomó este nuevo valor (3) para completar la expresión.
- La operación binaria final se redujo a la ecuación $3 + 3$, asignando un valor de 6 a la variable c .
- Una vez finalizada la sentencia mediante el punto y coma (`;`), el post-incremento latente de `a` se consolidó en la memoria, elevando su valor a 4. Los resultados fueron validados en pantalla mediante la función `printf`.

2. Validación de Jerarquía y Signos de Agrupación

En la segunda sección del programa, definí las variables de control $x = 10$ e $y = 3$ para contrastar los niveles de prioridad de los operadores de adición y multiplicación:

- **Expresión Lineal ($x + y * 2$):** Debido a que el operador `*` posee mayor precedencia que `+`, el compilador evaluó primero el subbloque $y * 2$ ($3 * 2 = 6$) y posteriormente sumó el valor de `x` ($10 + 6$), dando como resultado final 16.

- **Expresión Agrupada $(x + y) * 2$:** Al envolver los términos en paréntesis, se forzó al preprocesador y al compilador a romper la prioridad estándar. Se ejecutó en primera instancia la adición interna $x + y$ ($10 + 3 = 13$), y el subtotal fue multiplicado posteriormente por 2 (13×2), arrojando un resultado final de 26.

```
#include <stdio.h>

int main() {
    // Bloque 1: Operadores Unarios (Incrementos y Decrementos)
    int a = 3;
    int b = 4;
    int c = a++ + --b;

    printf("%d %d %d\n", a, b, c); // Salida esperada: 4 3 6

    // Bloque 2: Precedencia de Operadores y Paréntesis
    int x = 10;
    int y = 3;

    printf("%d\n %d\n", x + y * 2, (x + y) * 2); // Salidas: 16 y 26

    return 0;
}
```

CONCLUSIÓN

La resolución de esta práctica de laboratorio me permitió comprender con total claridad cómo piensa el compilador de C al enfrentarse a expresiones matemáticas que parecen simples a simple vista, pero que esconden un comportamiento lógico muy estricto. Confirmar de manera práctica que las hipótesis escritas en mis comentarios coincidían exactamente con la salida de la terminal consolidó mi aprendizaje sobre la gestión de variables.

Lo más valioso de la sesión fue analizar el ciclo de vida del post-incremento. Entender que el valor original de una variable puede quedarse "congelado" temporalmente para resolver una operación matemática (`a++`), mientras que el pre-decremento (`--b`) altera el valor de forma inmediata, cambia por completo la forma en que debo diseñar bucles y fórmulas en el futuro. Asimismo, comprobar el poder que tienen los paréntesis para romper la jerarquía estándar de multiplicación y suma me demuestra que el orden y la estructura sintáctica lo son todo en la programación estructurada. Esta actividad me deja una base matemática muy sólida para escribir expresiones complejas sin temor a generar comportamientos erráticos en el software.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Shields, H. (2019). *C Programming: A Modern Approach* (2nd ed.). W. W. Norton & Company.